

# Лабораторная работа № 4 «Case-классы и сопоставление с образцом в Scala»

15 апреля 2026 г.

Ольга Ивенкова, ИУ9-62Б

## Цель работы

Целью данной работы является приобретение навыков разработки case-классов на языке Scala для представления абстрактных синтаксических деревьев.

## Индивидуальный вариант

Регулярное выражение описано следующим абстрактным синтаксисом:

$\text{RegEx} \rightarrow \varepsilon \mid \text{SYMBOL} \mid \text{RegEx} \mid \text{RegEx} \cdot \text{RegEx} \mid \text{RegEx}^*$

Здесь  $\mid$  — объединение,  $\cdot$  — конкатенация.

Написать функцию `first : RegEx[T] => Set[Option(T)]`, вычисляющую множество FIRST для регулярного выражения. Параметр `T` — тип символов алфавита. Здесь `Some(ch)` представляет некоторый символ, `None` — пустую строку  $\varepsilon$ .

## Реализация

```
abstract class RegEx[T]

case class Epsilon[T]() extends RegEx[T]

case class Symbol[T](sym: T) extends RegEx[T]

case class Union[T](a: RegEx[T], b: RegEx[T]) extends RegEx[T]
case class Concat[T](a: RegEx[T], b: RegEx[T]) extends RegEx[T]

case class Star[T](a: RegEx[T]) extends RegEx[T]

object Main {
```

```

def first[T](re: RegEx[T]): Set[Option[T]] = re match {
  case _: Epsilon[T] =>
    Set(None)

  case Symbol(s) =>
    Set(Some(s))

  case Union(a, b) =>
    first(a) ++ first(b)

  case Concat(a, b) =>
    {
      val fa = first(a)
      if (fa contains None)
        (fa - None) ++ first(b)
      else
        fa
    }

  case Star(a) =>
    first(a) + None
}

def main(args: Array[String]): Unit = {
  val re = Concat(
    Star(Union(Symbol('a'),
      Symbol('b'))),
    Symbol('c')
  )

  println(s"(a|b)*.c")
  println(s"FIRST: ${first(re)}")

  val re2 = Union(Star(Symbol('x')), Epsilon[Char]())
  println(s"\n x*|ε")
  println(s"FIRST: ${first(re2)}")
}

```

## Тестирование

```

Set(Some('a'), Some('b'), Some('c'))
Set(Some('x'), None)

```

## **Вывод**

Приобретела навыки разработки case-классов на языке Scala для представления абстрактных синтаксических деревьев.